



គណៈកម្មាធិការ ជាតិ រើកម្ពុជា
ប្រាក់រៀល. ស្ថិតភាព. អភិវឌ្ឍន៍.

Implementation Guideline

KHQR SDK Documentation

Version 2.9

May 2025

DOCUMENT HISTORY

Version	Date	Description
1.0	14.12.2020	
1.0.1	04.01.2021	- Alignment content. - Remove the Error Handling part. - Make all contents in Requirement, Installation and Usage parts more consistent.
1.1	25.01.2021	- Add a timestamp to the QR code string. - Add md5 hash string to response data.
1.1.1	01.02.2021	- Add Bill Number, Store Label, Terminal Label in QR - Add the request parameter section.
1.2	17.02.2021	Adding decode KHQR information function.
1.3	15.03.2021	Add Javascript SDK
1.4	20.04.2021	- Add function to generate KHQR deeplink. - Rename decode function name from decodeQR to decode. (iOS only)
1.5	08.07.2021	Add deeplink routing diagram
1.6	11.07.2021	- Add merchant Id for merchant type. - Add Acquiring Bank merchant name for merchant type. - Add a mobile number to additional data. - Capture value of merchant city. - Update generate KHQR function. - Update decode response value. - Add deeplink routing diagram.
1.7	01.09.2021	- Update generate individual by adding new tag for account information and acquiring bank name. - Update decode function without validate on required fields.
1.8	12.11.2021	- Make improvements on the verify function for iOS, Java and javascript.
1.9	23.11.2021	- Fix an issue on generating KHQR of an amount with leading zero.
2.0	04.01.2021	Moving library for stable API request on javascript SDK and improve verify KHQR content function for all SDK.
2.1	13.01.2022	Fix bug on decode function in javascript SDK
2.2	28.03.2022	Fix bug on decode function in javascript SDK
2.3	20.04.2022	Fix an issue on the verify function from get invalid status to valid status when QR contains any optional subtags that are not defined in standard KHQR document.
2.4	05.09.2022	- Add changelogs on iOS SDK v 1.0.0.11 - Add a new error code.
2.5	20.02.2023	- Change package name from .org to .gov

		- Update phone number validation
2.6	19.05.2023	Remove example KHQR content that using main wallet
2.7	05.07.2023	- Add UPI tag - Add Purpose of Transaction tag - Add Merchant Alternate Language tag
2.8	24.02.2025	- Add expire timestamp on generic dynamic QR code - Add function decodeNonKhqr
2.9	06.05.2025	- Add Merchant Category Code (MCC)

TABLE CONTENTS

Introduction	6
Overview	6
Purpose	6
Scope	6
Version Control	7
Features	10
Requirement	11
iOS	11
Java	11
C#	11
Javascript	11
Installation	12
iOS	12
Java	13
C#	14
Javascript	16
Usage	18
iOS	20
Java	29
C#	34
Javascript	39
Deeplink Diagram	45
Standard Response	46
Response Format	46
Response Key and Code with definition	46
FAQ	48

Introduction

Overview

The standardization of KHQR code specification will help promote wider use of mobile retail payments in Cambodia and provide consistent user experience for merchants and consumers. It can enable interoperability in the payment industry. A common QR code would facilitate payments among different schemes, e-wallets and banks and would encourage small merchants to adopt KHQR code as payment method.

KHQR is created for retail or remittance in Cambodia and Cross-Border. It only requires a single QR for receiving transactions from any payment provider through Bakong including Bakong App. For more detail please refer to **Prakas KHQR Code Specification** in Cambodia.

Purpose

This document describes the detailed specification of how to use KHQR SDK offered by **National Bank of Cambodia**. The expected readers are NBC technical team and third-party technical team. This can be used as reference for any interest related to the KHQR SDK.

Scope

This document contains the complete description of the KHQR SDK specification including: Features, Requirement, Installation Guide, Usage and FAQ.

Version Control

SDK	Version	Description
Javascript	1.0.0	Having generate decode and verify KHQR feature
	1.0.1	Add generate deeplink feature
	1.0.2	- Ghost Version
	1.0.3	- Add merchant Id for merchant type. - Add Acquiring Bank merchant name for merchant type. - Add a mobile number to additional data. - Capture value of merchant city. - Update generate KHQR function. - Update decode response value.
	1.0.4	- Ghost version
	1.0.5	- Update generate individual by adding new tag for account information and acquiring bank name. - Update decode function without validate on required fields - Fix amount formatting
	1.0.6	- Adding a function to check bakong account
	1.0.7	Make improvements on the verify function.
	1.0.8	Moving library for stable API request
	1.0.9	Improve verify KHQR content function.
	1.0.10	Fixed on decode function (Return null data)
	1.0.11	Fixed on decode cache issue
	1.0.12	- Ghost version
	1.0.13	- Fixed on wrong decode information and issue with timestamp without subtag on decode function. - Fix null value on optional data does not throw exceptions anymore.
	1.0.14	Update phone number validation.
	1.0.15	- Added UnionPay Tags - Added merchant information-language template - Added purpose of transaction in AdditionalData Information

	1.0.18	- Decode Non KHQR - Adding timestamp
	1.0.20 (Latest)	- Add merchant category code, default 5999 - Add default value for Merchant City
iOS	0.1.1	
	1.0.0.1	- Add a timestamp to the QR code string. - Add md5 hash string to response data.
	1.0.0.2	- Add Bill number, Store label and terminal label to QR String.
	1.0.0.3	Add decode KHQR information function.
	1.0.0.4	Changes on function decode khqr - Validate length and CRC - Validate each tag according to KHQR specification
	1.0.0.5	Add generate deep link function
	1.0.0.6	- Add merchant Id for merchant type. - Add Acquiring Bank merchant name for merchant type. - Add a mobile number to additional data.. - Capture value of merchant city. - Update generate KHQR function. - Update decode response value.
	1.0.0.7	- Update generate individual by adding new tag for account information and acquiring bank name. - Update decode function without validate on required fields
	1.0.0.8	Make improvements on the verify function.
	1.0.0.9	Improve verify KHQR content function.
	1.0.0.10	Fix an issue on the verify function from get invalid status to valid status when QR contains any optional subtags that are not defined in standard KHQR document.
	1.0.0.11	1. Fixed wrong amount with precision 2. Validate tag length. 3. Validate duplicate tag.
	1.0.0.12	Change package name from .org to .gov
	1.0.0.13	Update phone number validation

	1.0.0.14	- Added UnionPay Tags - Added merchant information-language template - Added purpose of transaction in AdditionalData Information
	1.0.0.15	- Add a new error code when generating UPI account information with USD currency.
	1.0.0.16	- Add expire timestamp on generic dynamic QR code - Add function decodeNonKhqr
	1.0.0.17	- Add Merchant Category Code (MCC)
	1.0.0.18	- Change source repositories
	1.0.0.19 (Latest)	- Enhancement on verify function
Java	1.0.0.4	Add decode function
	1.0.0.5	Add generate deep link function
	1.0.0.6	- Add merchant Id for merchant type. - Add Acquiring Bank merchant name for merchant type. - Add a mobile number to additional data.. - Capture value of merchant city. - Update generate KHQR function. - Update decode response value.
	1.0.0.7	- Update generate individual by adding new tag for account information and acquiring bank name. - Update decode function without validate on required fields
	1.0.0.8	Make improvements on the verify function.
	1.0.0.9	Improve verify KHQR content function.
	1.0.0.10	- Change package name from .org to .gov - Update phone number validation
	1.0.0.11	- Added UnionPay Tags - Added merchant information-language template - Added purpose of transaction in AdditionalData Information
	1.0.0.12	- Add a new error-code when generating UPI account information with USD currency.
	1.0.0.13	- Update currency KHR validation on UPI
	1.0.0.14	- Add expiration timestamp tag

	1.0.0.15	- Add Merchant Category Code (MCC) - Add Function DecodeNonKHQR
	1.0.0.16 (Latest)	- Fix vulnerabilities dependencies
C#	0.1.1	
	1.0.0.1	- Add a timestamp to the QR code string. - Add md5 hash string to response data.
	1.0.0.2	- Add Bill number, Store label and terminal label to QR String.
	1.0.0.3	Add decode KHQR information function.
	1.0.0.4	Changes on function decode khqr - Validate length and CRC - Validate each tag according to KHQR specification
	1.0.0.5	Add generate deep link function
	1.0.0.6	- Add merchant Id for merchant type. - Add Acquiring Bank merchant name for merchant type. - Add a mobile number to additional data.. - Capture value of merchant city. - Update generate KHQR function. - Update decode response value.
	1.0.0.7	- Update generate individual by adding new tag for account information and acquiring bank name. - Update decode function without validate on required fields
	1.0.0.8	Fix an issue on generating KHQR of an amount with leading zero.
	1.0.0.9	Improve verify KHQR content function.
	1.0.0.10	Change package and namespace from org to gov
	1.0.0.11	Update phone number validation.
	1.0.0.12	- Added UnionPay Tags - Added merchant information-language template - Added purpose of transaction in AdditionalData Information
	1.0.0.13	Add a new error code when generating UPI account information with USD currency.
	1.0.0.14	Add expiration timestamp tag

	1.0.0.15 (Latest)	- Add Merchant Category Code (MCC) - Add Function DecodeNonKHQR
--	-------------------	--

Features

- Generate KHQR.
- Verification (Valid or Invalid).
- Decode KHQR Information.
- Generate KHQR Deeplink.
- Decode Non Khqr.

Requirement

iOS

- Development target 11.0 +
- Dependency management using Cocoapod

Java

- Source compatibility Java 8
- Target compatibility Java 8

C#

- NetStandard 2.0
- .Net Framework 4.0 and later
- .NET Core 2.0 and later
- Xamarin.iOS
- Xamarin.Android

Javascript

- NPM Javascript package management
- script raw file

Installation

iOS

CocoaPods

- How to

1. Add source to podfile

```
source  
"https://sambo-nbc:github_pat_11ASPB6YQ0D6RRw8ilETNw_zQ6WCqBs9RD0KNmfRLF4ig  
Nj7FflwBL3SAUbpYoobulE5DQXBOBBDRaoNcs@github.com/national-bank-of-cambodia/kh  
qr-ios-pod"
```

2. Add pod name to podfile

```
pod "BakongKHQR", '1.0.0.19'
```

3. Run command

```
pod install
```

Java

Maven

- How to

1. Add dependency to maven **pom.xml**

```
<dependency>
  <groupId>kh.gov.nbc.bakong_khqr</groupId>
  <artifactId>sdk-java</artifactId>
  <version>current_version</version>
</dependency>
```

Gradle

- **How to**

1. Add dependency to **build.gradle**

```
dependencies {
    implementation 'kh.gov.nbc.bakong_khqr:sdk-java:{current_version}'
}
```

C#

Nuget

- **How to**

1. Using Visual Studio Package Manager

```
Install-Package Kh.Gov.Nbc.BakongKHQR
```

2. Using Dotnet Cli

```
dotnet add package Kh.Gov.Nbc.BakongKHQR
```

3. Include in csproj

```
<ItemGroup>
  <PackageReference Include="Kh.Gov.Nbc.BakongKHQR" Version=current_version
/>
  <!-- ... other packages goes here -->
</ItemGroup>
```

Install Manually

If you prefer not to use nuget, you can integrate this SDK into your project manually via dll file.

- **How to**

1. Go to Nuget page of Kh.Gov.Nbc.BakongKHQR
<https://www.nuget.org/packages/Kh.Gov.Nbc.BakongKHQR>
2. Download package (nupkg file).
3. Extract the package using any unzip tool.
4. Copy the dll file based on your platform from the following folder.

For .NetFramework 4.0 copy dll from this path

```
lib/net40/kh.gov.nbc.bakong_khqr.dll
```

For .NetFramework 4.5 copy dll from this path

```
lib/net45/kh.gov.nbc.bakong_khqr.dll
```

For .NetFramework 4.6 copy dll from this path

```
lib/net46/kh.gov.nbc.bakong_khqr.dll
```

Other Platforms (.NetStandard 2.0) copy from this path

```
lib/netstandard2.0/kh.gov.nbc.bakong_khqr.dll
```

Please consult the NetStandard support table for further explanation.

<https://docs.microsoft.com/en-us/dotnet/standard/net-standard#net-implementation-support>

Javascript

NMP

- **How to**

1. Init project with NPM

```
npm init -y
```

2. Install KHQR SDK

```
npm install bakong-khqr
```

3. Use package

```
const {BakongKHQR, khqrData, IndividualInfo, MerchantInfo, SourceInfo} =  
require("bakong-khqr");
```

Or

```
import {BakongKHQR, khqrData, IndividualInfo, MerchantInfo, SourceInfo} from  
"bakong-khqr";
```

Raw Script

- **How to**

1. Link script to HTML

```
<script  
src="https://cdn.jsdelivr.net/gh/davidhuotkeo/bakong-khqr@master/khqr-1.0.20.min.  
js">\n</script>
```

2. Use it at .js file

```
let KHQR = BakongKHQR
// This is the object consist of BakongKHQR, khqrData, IndividualInfo, MerchantInfo,
SourceInfo
```

Usage

Request Parameter

- **Generate KHQR (Individual)**

Parameter	Type	Mandatory	Length	Description
BakongAccountID	String	Y	32	
AccountInformation	String	N	32	Account number or phone number.
AcquiringBank	String	N	32	Acquiring Bank name.
Currency	KHQRCurrency	N		Default: KHR
Amount	Double	N	13	
MerchantName	String	Y	25	
MerchantCity	String	N	15	Default: Phnom Penh
BillNumber	String	N	25	
MobileNumber	String	N	25	example: 855967854321
StoreLabel	String	N	25	
TerminalLabel	String	N	25	
PurposeOfTransaction	String	N	25	
UpiAccountInformation	String	N	31	example: 0001034400010344

				ABCDEFGHIJKLMNO which contain: Acquirer IIN: 00010344 Forwarding IIN: 00010344 Merchant ID: ABCDEFGHIJKLMNO
MerchantAlternateLangUagePreference	String	N	2	
MerchantNameAlternateLanguage	String	N	25	
MerchantCityAlternateLanguage	String	N	15	
ExpirationTimestamp	String	Y	13	Timestamp in milliseconds
MerchantCategoryCode	String	N	4	By default value 5999

- **Generate KHQR (Merchant)**

Parameter	Type	Mandatory	Length	Description
BakongAccountID	String	Y	32	
MerchantID	String	Y	32	
AcquiringBank	String	Y	32	Acquiring Bank name.
Currency	KHQRCurrency	N		Default: KHR
Amount	Double	N	13	
MerchantName	String	Y	25	
MerchantCity	String	N	15	Default: Phnom Penh
BillNumber	String	N	25	
MobileNumber	String	N	25	example: 855967854321
StoreLabel	String	N	25	
TerminalLabel	String	N	25	
PurposeOfTransaction	String	N	25	
UpiAccountInformation	String	N	31	example: 0001034400010344

				ABCDEFGHIJKLMNO which contain: Acquirer IIN: 00010344 Forwarding IIN: 00010344 Merchant ID: ABCDEFGHIJKLMNO
MerchantAlternateLanguagePreference	String	N	2	
MerchantNameAlternateLanguage	String	N	25	
ExpirationTimestamp	String	Y	13	Timestamp in milliseconds
MerchantCategoryCode	String	N	4	By default value 5999

- **Verification KHQR**

Parameter	Type	Mandatory	Length	Description
qrCode	String	Y		

- **Decode KHQR**

Parameter	Type	Mandatory	Length	Description
qrCode	String	Y		

- **KHQR Deeplink**

Parameter	Type	Mandatory	Length	Description
qr	String	Y		
url	String	Y		URL for API generate Deeplink. Provided by NBC team.
sourceInfo		N		
applconUrl	String	Y		Your app icon

appName	String	Y		Your app name
appDeepLinkCallBack	String	Y		Deeplink url for opening your app after payment is completed.

iOS

- Generate KHQR for Individual

Swift

```
let info = IndividualInfo(accountId: "john_smith@devb",
                           merchantName: "John Smith",
                           accountInformation: "85512233455",
                           acquiringBank: "Dev Bank",
                           currency: .Khr,
                           amount: 100)

info?.billNumber = "#12345"
info?.mobileNumber = "85512233455"
info?.storeLabel = "Coffee Shop"
info?.terminalLabel = "Cashier_1"
info?.purposeOfTransaction = "Morning coffee"
info?.upiAccountInformation = "12345678123456789012345"
info?.merchantAlternateLanguagePreference = "km"
info?.merchantNameAlternateLanguage = "ចិន ស្ទីន"
info?.merchantCityAlternateLanguage = "សៀមរាប"
info?.expirationTimestamp = NSNumber(value: 4101354327000)
info?.merchantCategoryCode = "0000"

let khqrResponse = BakongKHQR.generateIndividual(info!)

if khqrResponse.status?.code == 0 {
    let khqrData = khqrResponse.data as? KHQRData
    print("data: \(khqrData?.qr)")
    print("md5: \(khqrData?.md5)")
}
```

```
}
```

Objective C

```
IndividualInfo * info = [[IndividualInfo alloc] initWithAccountId: @"john_smith@devb"
                        merchantName: @"John Smith"
                        accountInformation: @"85512233455"
                        acquiringBank: @"Dev Bank"
                        currency: Khr
                        amount: 100];

[info setBillNumber: @"#12345"];
[info setMobileNumber: @"85512233455"];
[info setStoreLabel: @"Coffee Shop"];
[info setTerminalLabel: @"Cashier_1"];
[info setPurposeOfTransaction: @"Morning coffee"];
[info setUpiAccountInformation: @"12345678123456789012345"];
[info setMerchantAlternateLanguagePreference: @"km"];
[info setMerchantNameAlternateLanguage: @"ចន ស្ទីន"];
[info setMerchantCityAlternateLanguage: @"សៀមរាប"];
[info setExpirationTimestamp: @(4101354327000)];
[info setMerchantCategoryCode: @"0000"];

KHQRResponse * khqrResponse = [BakongKHQR generateIndividual: info];
if (khqrResponse.status.code == 0) {
    KHQRData * khqrData = (KHQRData *) khqrResponse.data;
    NSLog(@"data: %@", khqrData.qr);
    NSLog(@"md5: %@", khqrData.md5);
}
```

Result

```
data:
00020101021215231234567812345678901234529460015john_smith@devb01118551
22334550208Dev Bank52040000530311654031005802KH5910John Smith6010Phnom
Penh62710106#123450211855122334550311Coffee Shop0709Cashier_10814Morning
```

```
coffee64280002km0108ថ្ងៃ  
ស្ទឹង0206សៀមរាប9934001317393240684820113410135432700063042D66  
md5: d83fcf1a315a2ad45cb0aa86c96cd742
```

- **Generate KHQR for Merchant**

Swift

```
let merchantInfo = MerchantInfo(accountId: "khqr@devb",  
                                merchantId: "123456",  
                                merchantName: "John Smith",  
                                acquiringBank: "Dev Bank",  
                                currency: .Khr,  
                                amount: 100)  
  
merchantInfo?.billNumber = "#12345"  
merchantInfo?.mobileNumber = "85512233455"  
merchantInfo?.storeLabel = "Coffee Shop"  
merchantInfo?.terminalLabel = "Cashier_1"  
merchantInfo?.purposeOfTransaction = "Morning coffee"  
merchantInfo?.upiAccountInformation = "12345678123456789012345"  
merchantInfo?.merchantAlternateLanguagePreference = "km"  
merchantInfo?.merchantNameAlternateLanguage = "ថ្ងៃ ស្ទឹង"  
merchantInfo?.merchantCityAlternateLanguage = "សៀមរាប"  
merchantInfo?.expirationTimestamp = NSNumber(value: 4101354327000)  
merchantInfo?.merchantCategoryCiode = "0000"  
  
let khqrResponse = BakongKHQR.generateMerchant(merchantInfo!)  
  
if khqrResponse.status?.code == 0 {  
    let khqrData = khqrResponse.data as? KHQRData  
    print(khqrData?.qr)  
    print(khqrData?.md5)  
}
```

Objective C

```

MerchantInfo * merchantInfo = [[MerchantInfo alloc] initWithAccountId: @"khqr@devb"
merchantId: @"123456"
merchantName: @"John Smith"
acquiringBank: @"Dev Bank"
currency: Khr
amount: 100];

[merchantInfo setBillNumber: @"#12345"];
[merchantInfo setMobileNumber: @"85512233455"];
[merchantInfo setStoreLabel: @"Coffee Shop"];
[merchantInfo setTerminalLabel: @"Cashier_1"];
[merchantInfo setPurposeOfTransaction: @"Morning coffee"];
[merchantInfo setUpiAccountInformation: @"12345678123456789012345"];
[merchantInfo setMerchantAlternateLanguagePreferencel: @"km"];
[merchantInfo setMerchantNameAlternateLanguage @"ចិន ស្ទីន"];
[merchantInfo setMerchantCityAlternateLanguage: @"សៀមរាប"];
[merchantInfo setExpirationTimestamp: @(4101354327000)];
[merchantInfo setMerchantCategoryCode: @"0000"];

KHQRResponse * khqrResponse = [BakongKHQR generateMerchant: merchantInfo];

if (khqrResponse.status.code == 0) {
    KHQRData * khqrData2 = (KHQRData *) khqrResponse.data;
    NSLog(@"data: %@", khqrData2.qr);
    NSLog(@"md5: %@", khqrData2.md5);
}

```

Result

```

data:
00020101021215231234567812345678901234530350009khqr@devb010612345602
08Dev Bank52040000530311654031005802KH5910John Smith6010Phnom
Penh62710106#123450211855122334550311Coffee Shop0709Cashier_10814Morning
coffee64280002km0108ចិន
ស្ទីន0206សៀមរាប9934001317393247968240113410135432700063049922
md5: f54246baecc986d1cf58a39564094dd9

```

• Verification KHQR

Swift

```
let qrCode =
"00020101021215231234567812345678901234529460015john_smith@devb0111855
122334550208Dev bank5204599953038405406100.005802KH5910John
Smith6010Phnom Penh62710106#123450211855122334550311Coffee
Shop0709Cashier_10814Morning coffee64280002km0108ផ្លូវ
ស្ពឺន៍0206សៀមរាប99170013168800699761963042B5B"

let response = khqr?.verify(qrCode)

let crcValidation = response?.data as? CRCValidation

print("valid : \(crcValidation?.valid)")
```

Objective C

```
NSString *qrCode =
@"00020101021215231234567812345678901234529460015john_smith@devb01118
55122334550208Dev bank5204599953038405406100.005802KH5910John
Smith6010Phnom Penh62710106#123450211855122334550311Coffee
Shop0709Cashier_10814Morning coffee64280002km0108ផ្លូវ
ស្ពឺន៍0206សៀមរាប99170013168800699761963042B5B";

KHQRResponse * response = [khqr verify: qrCode];

CRCValidation* crcValidation = (CRCValidation *) response.data;

NSLog(@"valid: %d", crcValidation.valid);
```

Result

```
valid: 1
```

- Decode KHQR

Swift

```
let decodeResponse =  
  
BakongKHQR.decode("00020101021215231234567812345678901234530350009khqr  
@devb01061234560208Dev Bank52045999530311654031005802KH5910John  
Smith6010Phnom Penh62710106#123450211855122334550311Coffee  
Shop0709Cashier_10814Morning coffee64280002km0108ឆ្នាំ  
ស្ពឺន៍0206សៀមរាប993400131739324796824011341013543270006304ABDF")  
  
let decodeData = decodeResponse?.data as? KHQRDecodeData  
  
decodeData?.printAll()
```

Objective C

```
KHQRResponse* decodeResponse = [BakongKHQR decode:  
@"00020101021215231234567812345678901234530350009khqr@devb0106123456  
0208Dev Bank52045999530311654031005802KH5910John Smith6010Phnom  
Penh62710106#123450211855122334550311Coffee Shop0709Cashier_10814Morning  
coffee64280002km0108ឆ្នាំ  
ស្ពឺន៍0206សៀមរាប993400131739324796824011341013543270006304ABDF"];  
  
KHQRDecodeData* decodeData = (KHQRDecodeData *) decodeResponse.data;  
  
[decodeData printAll];
```

Result

```
payloadFormatIndicator = 01  
pointOfInitiationMethod = 12  
merchantType = 29  
upiAccountInformation = 12345678123456789012345  
bakongAccountID = john_smith@devb  
merchantAccountID = null  
accountInformation = 85512233455  
acquiringBank = Dev bank  
merchantCategoryCode = 5999  
countryCode = KH  
merchantName = John Smith
```

```

merchantCity = Phnom Penh
transactionCurrency = 116
transactionAmount = 100
billNumber = #12345
mobileNumber = 85512233455
storeLabel = Coffee Shop
terminalLabel = Cashier_1
purposeOfTransaction = Morning coffee
timestamp = 131688006997619
merchantAlternateLanguagePreference = km
merchantNameAlternateLanguage = ធីន ស្ទីន
merchantCityAlternateLanguage = សៀមរាប
expirationTimestamp = 4101354327000
crc = 2B5B

```

- **Decode Non Khqr**

Swift

```

let khqrResponse =

BakongKHQR.decodeNonKhqr("00020101021215231234567812345678901234530350
009khqr@devb01061234560208Dev Bank52045999530311654031005802KH5910John
Smith6010Phnom Penh62710106#123450211855122334550311Coffee
Shop0709Cashier_10814Morning coffee64280002km0108ធីន
ស្ទីន0206សៀមរាប993400131739324796824011341013543270006304ABDF")

if khqrResponse?.status?.code == 0 {

    let decodeData = khqrResponse?.data as? KHQREMVQRData

    print("data: \(decodeData)")

}

```

Objective C

```

KHQRResponse * khqrResponse = [BakongKHQR decodeNonKhqr:
@"00020101021215231234567812345678901234530350009khqr@devb0106123456
0208Dev Bank52045999530311654031005802KH5910John Smith6010Phnom
Penh62710106#123450211855122334550311Coffee Shop0709Cashier_10814Morning
coffee64280002km0108ធីន
ស្ទីន0206សៀមរាប993400131739324796824011341013543270006304ABDF"];

if (khqrResponse.status.code == 0) {

    KHQREMVQRData* decodeData = (KHQREMVQRData *) khqrResponse.data;

```

```
    NSLog(@"data: %@", decodeData);  
}
```

Result

```
data: {  
  00 = 01;  
  01 = 12;  
  15 = 12345678123456789012345;  
  30 = {  
    00 = "khqr@devb";  
    01 = 123456;  
    02 = "Dev Bank";  
  };  
  52 = 5999;  
  53 = 116;  
  54 = 100;  
  58 = KH;  
  59 = "John Smith";  
  60 = "Phnom Penh";  
  62 = {  
    01 = "#12345";  
    02 = 85512233455;  
    03 = "Coffee Shop";  
    07 = "Cashier_1";  
    08 = "Morning coffee";  
  };  
  63 = ABDF;  
  64 = {  
    00 = km;  
    01 = "\U1785\U1793 \U179f\U17d2\U1798\U17b8\U1793";  
    02 = "\U179f\U17c0\U1798\U179a\U17b6\U1794";  
  };  
  99 = {  
    00 = 1739324796824;  
    01 = 4101354327000;  
  };
```



```
}
```

- **Generate KHQR deeplink**

Swift

```
let deeplinkResponse =  
    BakongKHQR.generateDeepLink("http://api.example.com/v1/generate_deeplink_by_qr",  
  
    qr:"00020101021229180014jonhsmith@nbcq52045999530384054031.05802KH5910Jo  
    nh Smith6010Phnom Penh6304C297",  
  
        sourceInfo: SourceInfo(applconURL:"http://cdn.example.com/icons.logo.png",  
                                appName: "Example App",  
                                appDeepLinkCallBack: "http://app.example.com")  
    )  
if deeplinkResponse.status.code == 0 {  
    let deeplinkData = deeplinkResponse?.data as? KHQRDeepLinkData  
    print(deeplinkData?.shortLink)  
} else {  
    print(deeplinkResponse.status.message)  
}
```

Objective C

```
KHQRResponse* deeplinkResponse = [BakongKHQR generateDeepLink:  
    @"http://api.example.com/v1/generate_deeplink_by_qr"  
  
    qr:  
    @"00020101021229180014jonhsmith@nbcq52045999530384054031.05802KH5910Jo  
    nh Smith6010Phnom Penh6304C297"  
  
    sourceInfo: [[SourceInfo alloc]  
        initWithApplconURL:@"http://cdn.example.com/icons.logo.png"  
        appName:@"Example App"  
        appDeepLinkCallBack:@"http://app.example.com"]  
    ];
```

```

if (deeplinkResponse.status.code == 0) {
    KHQRDeepLinkData* deeplinkData = (KHQRDeepLinkData *) deeplinkResponse.data;
    NSLog(@"%@", deeplinkData.shortLink);
} else {
    NSLog(@"%@",deeplinkResponse.status.message);
}

```

Result

<https://bakongsit.page.link/pBJzebGwUuQfMfhf9>

***** Note:** Generated short links will be different every time method is called and also depend on which api url being used.

Java

- **Generate KHQR for Individual**

```

IndividualInfo individualInfo = new IndividualInfo();
individualInfo.setBakongAccountId("john_smith@devb");
individualInfo.setAccountInformation("85512233455");
individualInfo.setAcquiringBank("Dev Bank");
individualInfo.setCurrency(KHQRCurrency.KHR);
individualInfo.setAmount(100.0);
individualInfo.setMerchantName("John Smith");
individualInfo.setMerchantCity("PHNOM PENH");
individualInfo.setBillNumber("#12345");
individualInfo.setMobileNumber("85512233455");
individualInfo.setStoreLabel("Coffee Shop");
individualInfo.setTerminalLabel("Cashier_1");
individualInfo.setUpiAccountInformation("1234567812345678ABCDEFGHJKLMNO");
individualInfo.setPurposeOfTransaction("Buy coffee");
individualInfo.setMerchantAlternateLanguagePreference("km");
individualInfo.setMerchantNameAlternateLanguage("កាហ្វេ ស្តារ");
individualInfo.setMerchantCityAlternateLanguage("ភ្នំពេញ");
individualInfo.setExpirationTimestamp(4101354327000L);

```

```

individualInfo.setMerchantCategoryCode("1234");

KHQRResponse<KHQRData> response = BakongKHQR.generateIndividual(individualInfo);
if (response.getKHQRStatus().getCode() == 0) {
    System.out.println("data: " + response.getData().getQr());
    System.out.println("md5: " + response.getData().getMd5());
}

```

Result

```

data:
00020101021215311234567812345678ABCDEFGHJKLMNO29460015john_smith@dev
b0111855122334550208Dev Bank52041234530311654031005802KH5910John
Smith6010PHNOM PENH62670106#123450211855122334550311Coffee
Shop0709Cashier_10810Buy coffee64290002km0108កម្ពុជា
ស្ម័គ្រចិត្ត0207កម្ពុជា993400131745915488053011341013543270006304FD7B

md5: 9dc0fe3e9ea50b99ddccd47055333ed8

```

- **Generate KHQR for Merchant**

```

MerchantInfo merchantInfo = new MerchantInfo();
merchantInfo.setBakongAccountId("khqr@devb");
merchantInfo.setMerchantId("123456");
merchantInfo.setAcquiringBank("Dev Bank");
merchantInfo.setCurrency(KHQRCurrency.KHR);
merchantInfo.setAmount(100.0);
merchantInfo.setMerchantName("John Smith");
merchantInfo.setMerchantCity("PHNOM PENH");
merchantInfo.setBillNumber("#12345");
merchantInfo.setMobileNumber("85512233455");
merchantInfo.setStoreLabel("Coffee Shop");
merchantInfo.setTerminalLabel("Cashier_1");
merchantInfo.setUpiAccountInformation("1234567812345678ABCDEFGHJKLMNO");
merchantInfo.setPurposeOfTransaction("Buy coffee");
merchantInfo.setMerchantAlternateLanguagePreference("km");

```

```

merchantInfo.setMerchantNameAlternateLanguage("ចំនួន ស្ទីន");
merchantInfo.setMerchantCityAlternateLanguage("ភ្នំពេញ");
merchantInfo.setExpirationTimestamp(4101354327000L);
merchantInfo.setMerchantCategoryCode("1234");

KHQRResponse<KHQRData> response = BakongKHQR.generateMerchant(merchantInfo);
if (response.getKHQRStatus().getCode() == 0) {
    System.out.println("data: " + response.getData().getQr());
    System.out.println("md5: " + response.getData().getMd5());
}

```

Result

```

data:
00020101021215311234567812345678ABCDEFGHJKLMNO30350009khqr@devb0106
1234560208Dev Bank52041234530311654031005802KH5910John Smith6010PHNOM
PENH62670106#123450211855122334550311Coffee Shop0709Cashier_10810Buy
coffee64290002km0108ចំនួន
ស្ទីន0207ភ្នំពេញ993400131745915556891011341013543270006304594D
md5: 2d32b8a97fe5d4ff091405b166b22d5b

```

• Verification KHQR

```

String qrCode =
"00020101021215311234567812345678ABCDEFGHJKLMNO30350009khqr@devb010
61234560208Dev Bank52041234530311654031005802KH5910John
Smith6010PHNOM PENH62670106#123450211855122334550311Coffee
Shop0709Cashier_10810Buy coffee64290002km0108ចំនួន
ស្ទីន0207ភ្នំពេញ993400131745915556891011341013543270006304594D";

KHQRResponse<CRCValidation> response = BakongKHQR.verify(qrCode);
System.out.println("valid: " + response.getData().isValid());

```

Result

```

valid: true

```

- Decode KHQR

```
String qrCode =
"00020101021215311234567812345678ABCDEFGHJKLMNO30350009khqr@devb010
61234560208Dev Bank52041234530311654031005802KH5910John
Smith6010PHNOM PENH62670106#123450211855122334550311Coffee
Shop0709Cashier_10810Buy coffee64290002km0108ផ្សំ
ស្ពីន៍0207ភ្នំពេញ993400131745915556891011341013543270006304594D";

KHQRResponse<KHQRDecodeData> response = BakongKHQR.decode(qrCode);

System.out.println(response);
```

Result

```
KHQRResponse{
  status=KHQRStatus{
    code=0,
    errorCode=null,
    message='null'
  }
  data=KHQRDecodeData{
    payloadFormatIndicator='01',
    pointOfInitiationMethod='12',
    merchantType='30',
    bakongAccountID='khqr@devb',
    merchantId='123456',
    accountInformation='null',
    upiAccountInformation='1234567812345678ABCDEFGHJKLMNO'
    acquiringBank='Dev Bank',
    merchantCategoryCode='1234',
    countryCode='KH',
    merchantName='John Smith',
    merchantCity='PHNOM PENH',
    transactionCurrency='116',
    transactionAmount='100',
    billNumber='#12345',
```

```

mobileNumber='85512233455',
storeLabel='Coffee Shop',
terminalLabel='Cashier_1',
purposeOfTransaction='Buy coffee',
merchantAlternateLanguagePreference='km'
merchantNameAlternateLanguage='កាហ្វេ ស្ទីន'
merchantCityAlternateLanguage='ភ្នំពេញ'
creationTimestamp='1745915556891',
expirationTimestamp='4101354327000',
crc='594D'
}
}

```

- **Decode Non KHQR**

```

String qrCode =
"00020101021215311234567812345678ABCDEFGHJKLMNO30350009khqr@devb010
61234560208Dev Bank52041234530311654031005802KH5910John
Smith6010PHNOM PENH62670106#123450211855122334550311Coffee
Shop0709Cashier_10810Buy coffee64290002km0108កាហ្វេ
ស្ទីន0207ភ្នំពេញ993400131745915556891011341013543270006304594D";

KHQRResponse<KHQRHashMap> response =
BakongKHQR.decodeNonKHQR(qrCode);

System.out.println(response.getData());

```

Result

```

{
  00 = 01,
  01 = 12,
  15 = 1234567812345678ABCDEFGHJKLMNO,
  30 = {
    00 = khqr@devb,
    01 = 123456,
    02 = Dev Bank,
  }
  52 = 1234,
  53 = 116,

```

```

54 = 100,
58 = KH,
59 = John Smith,
60 = PHNOM PENH,
62 = {
    01 = #12345,
    02 = 85512233455,
    03 = Coffee Shop,
    07 = Cashier_1,
    08 = Buy coffee,
}
63 = 594D,
64 = {
    00 = km,
    01 = ចិន ស្ទីន,
    02 = ភ្នំពេញ,
}
99 = {
    00 = 1745915556891,
    01 = 4101354327000,
}
}

```

- **Generate KHQR deeplink**

```

String url = "http://api.example.com/v1/generate_deeplink_by_qr";
String qr =

"00020101021229180014jonhsmith@nbcq52045999530384054031.05802KH5910Jonh
Smith6010Phnom Penh6304C297";

SourceInfo sourceInfo = new SourceInfo();
sourceInfo.setAppName("Example App");
sourceInfo.setAppIconUrl("http://cdn.example.com/icons.logo.png");
sourceInfo.setAppDeepLinkCallback("http://app.example.com");
KHQRResponse<KHQRDeepLinkData> response = BakongKHQR.generateDeepLink(url,
qr, sourceInfo);

```

```

if (response.getKHQRStatus().getCode() == 0) {
    System.out.println(response.getData().getShortLink());
}else {
    System.out.println(response.getKHQRStatus().getMessage());
}

```

Result

<https://bakongsit.page.link/8dBnac2EQefMtmsM6>

*** **Note:** Generated short links will be different every time method is called and also depend on which api url being used.

C#

- **Generate KHQR For Individual**

```

var response = BakongKHQR.GenerateIndividual(
    new IndividualInfo {
        BakongAccountID = "john_smith@devb",
        Currency = KHQRCurrency.KHR,
        AccountInformation = "85512233455",
        AcquiringBank = "Dev Bank",
        Amount = 10000,
        MerchantName = "John Smith",
        MerchantCity = "PHNOM PENH",
        BillNumber = "#12345",
        MobileNumber = "85512233455",
        StoreLabel = "Coffee Shop",
        TerminalLabel = "Cashier_1",
        UpiAccountInformation = "1234567812345678ABCDEFGHIJKLMNO",
        PurposeOfTransaction = "Buy coffee",
        MerchantAlternateLanguagePreference = "km",
    }
);

```



```

MerchantNameAlternateLanguage = "ចំនួន ស្តី",
MerchantCityAlternateLanguage = "ភ្នំពេញ",
ExpirationTimestamp = 4101354327000,
MerchantCategoryCode = "1234"

});
if (response.Status.Code == 0){
    Console.WriteLine("data: " + response.Data.QR);
    Console.WriteLine("md5: " + response.Data.MD5);
}

```

Result

```

data:
00020101021215311234567812345678ABCDEFGHJKLMNO29460015john_smith@dev
b0111855122334550208Dev Bank5204123453031165405100005802KH5910John
Smith6010PHNOM PENH62670106#123450211855122334550311Coffee
Shop0709Cashier_10810Buy coffee64290002km0108ចំនួន
ស្តី0207ភ្នំពេញ993400131745913986332011341013543270006304D83C
md5: d73232b3d6aced9164241401ed0812c6

```

• Generate KHQR For Merchant

```

var response = BakongKHQR.GenerateMerchant( new MerchantInfo
{
    BakongAccountID = "khqr@devb",
    MerchantID = "123456",
    AcquiringBank = "Dev Bank",
    Currency = KHQRCurrency.KHR,
    Amount = 10000,
    MerchantName = "John Smith",
    MerchantCity = "PHNOM PENH",
    BillNumber = "#12345",
    MobileNumber = "85512233455",
    StoreLabel = "Coffee Shop",
    TerminalLabel = "Cashier_1",
    UpiAccountInformation = "1234567812345678ABCDEFGHJKLMNO",
    PurposeOfTransaction = "Buy coffee",
    MerchantAlternateLanguagePreference = "km",
    MerchantNameAlternateLanguage = "ចំនួន ស្តី",
}

```

```

MerchantCityAlternateLanguage = "ភ្នំពេញ",
ExpirationTimestamp = 4101354327000,
MerchantCategoryCode = "1234"
});
if (response.Status.Code == 0){
    Console.WriteLine("data: " + response.Data.QR);
    Console.WriteLine("md5: " + response.Data.MD5);
}

```

Result

```

data:
00020101021215311234567812345678ABCDEFGHJKLMNO30350009khqr@devb0106
1234560208Dev Bank5204123453031165405100005802KH5910John
Smith6010PHNOM PENH62670106#123450211855122334550311Coffee
Shop0709Cashier_10810Buy coffee64290002km0108ធីន
ស្ទីន0207ភ្នំពេញ993400131745914132168011341013543270006304207D
md5: 07916c8d970b8ad0492f7510b500c74b

```

• Verification KHQR

```

var response =
BakongKHQR.Verify("00020101021215311234567812345678ABCDEFGHJKLMNO3035
0009khqr@devb01061234560208Dev
Bank5204599953031165405100005802KH5910John Smith6010PHNOM
PENH62670106#123450211855122334550311Coffee Shop0709Cashier_10810Buy
coffee64290002km0108ធីន
ស្ទីន0207ភ្នំពេញ9934001317393307745630113410135432700063043AC3");
if (response.Status.Code == 0)
{
    Console.WriteLine("valid: " + response.Data.Valid);
}

```

Result:

```
valid: True
```

• Decode KHQR

```

var response =
BakongKHQR.Decode("00020101021215311234567812345678ABCDEFGHJKLMNO303

```

```

50009khqr@devb01061234560208Dev
Bank5204599953031165405100005802KH5910John Smith6010PHNOM
PENH62670106#123450211855122334550311Coffee Shop0709Cashier_10810Buy
coffee64290002km0108ផ្លូវ
ស្ពឺន0207ភ្នំពេញ9934001317393307745630113410135432700063043AC3");

var options = new JsonSerializerOptions()
{
    WriteIndented = true
};

Console.WriteLine(JsonSerializer.Serialize(response, options));

```

Result

```

{
  "Status": {
    "Code": 0,
    "ErrorCode": null,
    "Message": null
  },
  "Data": {
    "PayloadFormatIndicator": "01",
    "PointOfInitiationMethod": "12",
    "MerchantType": "30",
    "BakongAccountID": "khqr@devb",
    "MerchantID": "123456",
    "AccountInformation": null,
    "UpiAccountInformation": "1234567812345678ABCDEFGHJKLMNO",
    "AcquiringBank": "Dev Bank",
    "MerchantCategoryCode": "1234",
    "CountryCode": "KH",
    "MerchantName": "John Smith",
    "MerchantCity": "PHNOM PENH",
    "TransactionCurrency": "116",
    "TransactionAmount": "10000",
    "BillNumber": "#12345",
    "MobileNumber": "85512233455",
    "StoreLabel": "Coffee Shop",
    "TerminalLabel": "Cashier_1",

```

```

    "PurposeOfTransaction": "Buy coffee",
    "MerchantAlternateLanguagePreference": "km",
    "MerchantNameAlternateLanguage": "ចិន ស្ទីន",
    "MerchantCityAlternateLanguage": "ភ្នំពេញ",
    "CreationTimestamp": "1739330774563",
    "ExpirationTimestamp": "4101354327000",
    "CRC": "3AC3"
  }
}

```

- **Decode Non KHQR**

```

var response =
BakongKHQR.DecodeNonKHQR("0002010102111523123456781234567890123453040
0014jonhsmith@nbcq01061234560208Dev Bank5204599953031165802KH5910Jonh
Smith6010Phnom Penh9917001316873168216096310abcdef");

var options = new JsonSerializerOptions()
{
    WriteIndented = true
};

Console.WriteLine(JsonSerializer.Serialize(response, options));

```

Result

```

{
  "Status": {
    "Code": 0,
    "ErrorCode": null,
    "Message": null
  },
  "Data": {
    "00": "01",
    "01": "11",
    "15": "12345678123456789012345",
    "30": {
      "00": "jonhsmith@nbcq",
      "01": "123456",
      "02": "Dev Bank"
    }
  }
}

```

```

    },
    "52": "5999",
    "53": "116",
    "58": "KH",
    "59": "Jonh Smith",
    "60": "Phnom Penh",
    "99": {
        "00": "1687316821609"
    }
}
}
}

```

- **Generate KHQR deeplink**

```

var url = "http://api.example.com/v1/generate_deeplink_by_qr";

var qr =
"00020101021229180014jonhsmith@nbcq52045999530384054031.05802KH5910Jonh
Smith6010Phnom Penh6304C297";

var sourceInfo = new SourceInfo {
    AppName = "Example App",
    AppIconUrl = "http://cdn.example.com/icons.logo.png",
    AppDeepLinkCallback = "http://app.example.com"
};

var response = BakongKHQR.GenerateDeepLink(url, qr, sourceInfo);
if (response.Status.Code == 0) {
    Console.WriteLine(response.Data.ShortLink);
} else {
    Console.WriteLine(response.Status.Message);
}

```

Result

```
https://bakongsit.page.link/DqKPTPQ521gXd4g8A
```

***** Note:** Generated short links will be different every time method is called and also depend on which api url being used.

Javascript

- Generate Individual KHQR

```
const {
  BakongKHQR,
  khqrData,
  IndividualInfo,
  MerchantInfo,
} = require("bakong-khqr");

const optionalData = {
  currency: khqrData.currency.khr,
  amount: 10000,
  merchantCategoryCode: "5999",
  mobileNumber: "85512233455",
  storeLabel: "Coffee Shop",
  terminalLabel: "Cashier_1",
  purposeOfTransaction: "oversea",
  languagePreference: "km",
  merchantNameAlternateLanguage: "ចិន ស្ទីន",
  merchantCityAlternateLanguage: "សៀមរាប",
  upiMerchantAccount: "0001034400010344ABCDEFGHJKLMNO",
};

const individualInfo = new IndividualInfo(
```

```

    "jonh_smith@devb",
    "Jonh Smith",
    "PHNOM PENH",
    optionalData);

const KHQR = new BakongKHQR();

const individual = KHQR.generateIndividual(individualInfo);

console.log("qr: " + individual.data.qr);

console.log("md5: " + individual.data.md5);

```

Result

```

qr:
'00020101021215310001034400010344ABCDEFGHJKLMNOP29140010devit@pras520
4599953031165405100005802KH5910Jonh
Smith6010Battambang62540211855122334550311Coffee
Shop0709Cashier_10807oversea64280002km0108ធុន
ស្ពឺន0206សៀមរាប991700131700975290649630469A6'

md5: 'aaab846681f1e849ca525d6e5c039f10'

```

- **Generate Merchant KHQR**

```

const {
    BakongKHQR,
    khqrData,
    IndividualInfo,
    MerchantInfo,
} = require("bakong-khqr");

const optionalData = {
    currency: khqrData.currency.usd,
    amount: 100.5,
    merchantCategoryCode: "5999",
    mobileNumber: "85512233455",
    storeLabel: "Coffee Shop",

```

```

terminalLabel: "Cashier_1",
purposeOfTransaction: "oversea",
languagePreference: "km",
merchantNameAlternateLanguage: "ចិន ស្ទីន",
merchantCityAlternateLanguage: "សៀមរាប",
upiMerchantAccount: "0001034400010344ABCDEFGHJKLMNOP"
};

const merchantInfo = new MerchantInfo(
    "khqr@devb",
    "Jonh Smith",
    "PHNOM PENH",
    "123456",
    "Dev Bank",
    optionalData
);

const KHQR = new BakongKHQR();
const merchant = KHQR.generateMerchant(merchantInfo);
console.log("qr: " + merchant.data.qr);
console.log("md5: " + merchant.data.md5);

```

Result

```

qr:
'00020101021215310001034400010344ABCDEFGHJKLMNOP30350009khqr@devb010
61234560208Dev Bank52045999530311654061000005802KH5910Jonh
Smith6010PHNOM PENH62540211855122334550311Coffee
Shop0709Cashier_10807oversea64280002km0108ចិន
ស្ទីន0206សៀមរាប9917001317009754510736304B8E7',
md5: 'd16b8758f53c246a58c650f91e430d2f'

```

• Verification KHQR

```

let khqrString = "00020101021230350009khqr@devb01061234560208Dev
Bank5204599953038405406100.505802KH5910Jonh Smith6010PHNOM
PENH62430211855122334550311Coffee
Shop0709Cashier_19917001316844802622846304009F";
const isKHQR = BakongKHQR.verify(khqrString);

```



```
console.log("valid:" + isKHQR.isValid)
```

Result

```
valid:true
```

- **Decode KHQR**

```
const KHQRString =  
  
"00020101021215311234567812345678ABCDEFGHJKLMNO30350009khqr@devb010  
61234560208Dev Bank5204599953031165405100005802KH5910John  
Smith6010PHNOM PENH62670106#123450211855122334550311Coffee  
Shop0709Cashier_10810Buy coffee64290002km0108ឆ្នាំ  
ស៊ីន0207ភ្នំពេញ9934001317393307745630113410135432700063043AC3";  
  
const decodeResult = BakongKHQR.decode(KHQRString);  
  
console.log(decodeResult);
```

Result

```
{  
  status: { code: 0, errorCode: null, message: null },  
  data: {  
    merchantType: '30',  
    bakongAccountID: 'khqr@devb',  
    accountInformation: null,  
    merchantID: '123456',  
    acquiringBank: 'Dev Bank',  
    billNumber: '#12345',  
    mobileNumber: '85512233455',  
    storeLabel: 'Coffee Shop',  
    terminalLabel: 'Cashier_1',
```



```

'53': '840',
'54': '500.0',
'58': 'KH',
'59': 'John Smith',
'60': 'Phnom Penh',
'62': {
  '01': 'Invoice#069',
  '03': 'Coffee Khlaing',
  '07': 'Cooooooooooooooooooooooooounter 2'
},
'63': '58D8',
'99': { '00': '1727340564907', '01': '1727340624907' },
'00': '01',
'01': '12'
}
}

```

- **Generate KHQR deeplink**

```

const khqrString =
"00020101021229190015john_smith@devb5204599953038405405100.05802KH5910J
ohn Smith6010Phnom Penh6304BF30";

const sourceInfo = new SourceInfo(
  "http://cdn.example.com/icons.logo.png",
  "Example App",
  "http://app.example.com");

const deeplink = khqr.generateDeepLink(
  "http://api.example.com/v1/generate_deeplink_by_qr",
  khqrString,
  sourceInfo
);

deeplink.then((data) => {
  if (data.status.code == 0) {
    console.log(data.data.shortLink);
  } else {

```

```

console.log(data.status.message);

}

});

```

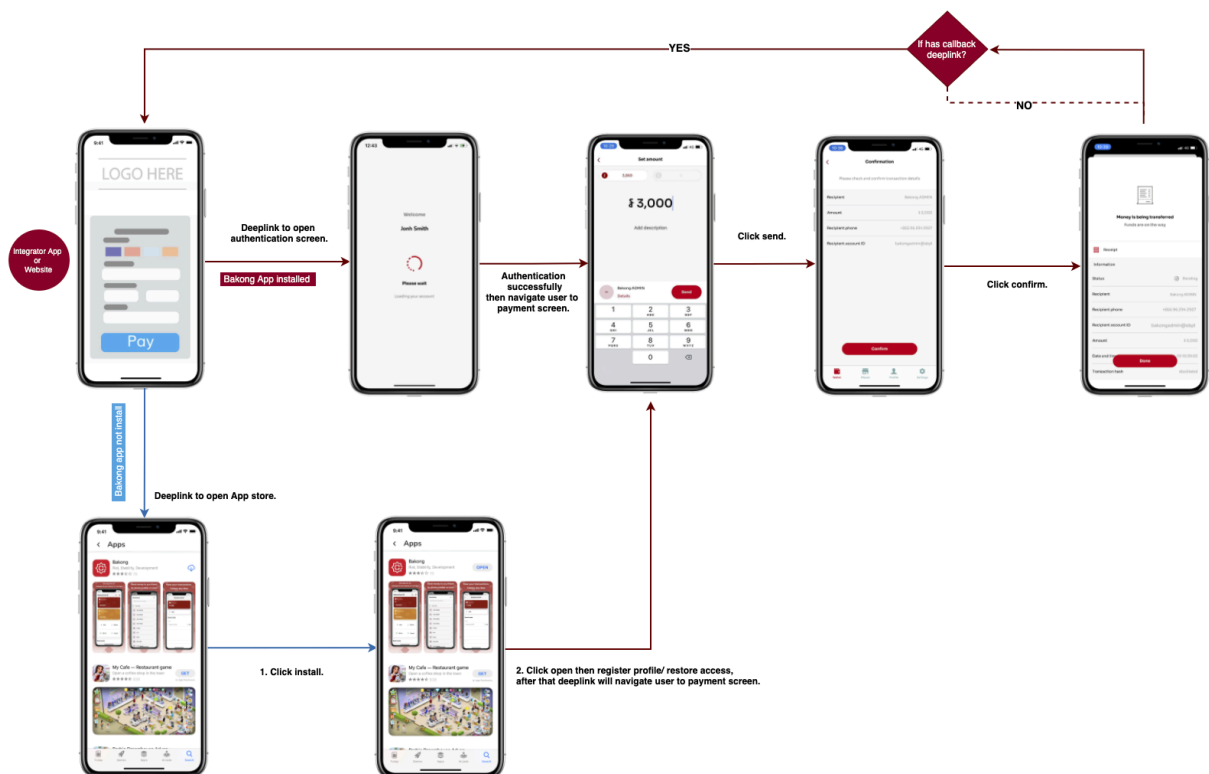
Result

<https://bakongsit.page.link/16ahB249Xu9TQc676>

*** **Note:** Generated short links will be different every time method is called and also depend on which api url being used.

Deeplink Diagram

Deeplink Rounting



Standard Response

Response Format

```
{
  "status": {
    "code": 0,
    "errorCode": null,
    "message": 'This is message'
  },
  "data": {
    //response based on each function
    qr: "00020101021229460015john_smith@devb0111855122334550208Dev
Bank52045999530384054031005802KH5910John Smith6010Phnom
Penh62500106#123450211855122334550311Coffee
Shop07060123459917001316311703855496304980C"
  }
}
```

Response Key and Code with definition

Key	Type	Value	Message
code	int		
		0	Success
		1	Failed
errorCode	int		
		1	Bakong Account ID cannot be null or empty
		2	Merchant name cannot be null or empty
		3	Bakong Account ID is invalid
		4	Amount is invalid
		5	Merchant type cannot be null or empty
		6	Bakong Account ID Length is invalid
		7	Merchant Name Length is invalid
		8	KHQR provided is invalid

		9	Currency type cannot be null or empty
		10	Bill Number Length is invalid
		11	Store Label Length is invalid
		12	Terminal Label Length is invalid
		13	Cannot reach Bakong Open API service. Please check internet connection
		14	Source Info for Deep Link is invalid
		15	Internal Server Error
		16	Payload Format Indicator Length is invalid
		17	Point of Initiation Length is invalid
		18	Merchant Category Length is invalid
		19	Transaction Currency Length is invalid
		20	Country Code Length is invalid
		21	Merchant City Length is invalid
		22	CRC Length is invalid
		23	Payload Format Indicator cannot be null or empty
		24	CRC cannot be null or empty
		25	Merchant Category cannot be null or empty
		26	Country Code cannot be null or empty
		27	Merchant City cannot be null or empty
		28	Unsupported currency
		29	Deep Link URL is not valid
		30	Merchant ID cannot be null or empty
		31	Acquiring Bank cannot be null or empty
		32	Merchant ID Length is invalid
		33	Acquiring Bank Length is invalid
		34	Mobile Number Length is invalid
		35	Tag not in order

		36	Account information length is invalid
		37	Merchant Alternate Language Preference cannot be null or empty
		38	Merchant Alternate Language Preference Length is invalid
		39	Merchant Name Alternate Language cannot be null or empty
		40	Merchant Name Alternate Language Length is invalid
		41	Merchant City Alternate Language Length is invalid
		42	Purpose of Transaction Length is invalid
		43	Upi Account Information Length is invalid
		44	KHQR does not support UPI Account Information with USD currency
		45	Expiration timestamp is required for dynamic KHQR
		46	This dynamic KHQR has expired
		47	This dynamic KHQR has invalid field transaction amount
		48	Point of Initiation Method is invalid
		49	Expiration timestamp length is invalid
		50	Expiration timestamp is in the past
		51	Invalid Merchant Category Code
message	String		The message describes the result.
data	Any Object		The response will be different based on function
qr	String		emv qr code provided by KHQR library.
md5	String		hash value of bakong KHQR to verify transaction status.

FAQ

1. If my bank wants to know more or inquire about KHQR, who should I talk to?

KHQR team of Association of Bank in Cambodia

2. If my bank wants to know more or inquire about How To Implement KHQR By Using Bakong As a Payment Switch, who should I talk to?

Bakong team of National Bank of Cambodia